HTTP QUERY Method:
A Security-Focused Literature Survey and Analysis

by

Anish Shobith

Registration Number

M.E. CyberSecurity

Under the guidance of

Panel Member 1:

Name

Designation

Manipal School of Information Sciences

MAHE, Manipal

Panel Member 2:

Name

Designation

Manipal School of Information Sciences

MAHE, Manipal

Presentation Outline

1. Introduction: HTTP methods, URIs and the gap between GET and POST
2. Objectives
3. Literature Review: standards and studies
4. Description: the QUERY method, code and security
5. Conclusion

References

An HTTP request and its response

a GET exchange

```
GET /products?category=laptop HTTP/1.1 (1)
Host: shop.example
Accept: application/json
Authorization: Bearer eyJhbGciOiJFUzI1NiJ9… (2)

HTTP/1.1 200 OK (3)
Content-Type: application/json
Cache-Control: max-age=60

{ "results": [ { "id": 7, "name": "ThinkPad X1" }
```

1. **Method and target URI.** The query is in the URI.
2. **Headers.** Here: the format and the credentials.
3. **Status line.** The result of the request.
4. **Content.** A GET request has none [RFC 9110 §9.3.1]

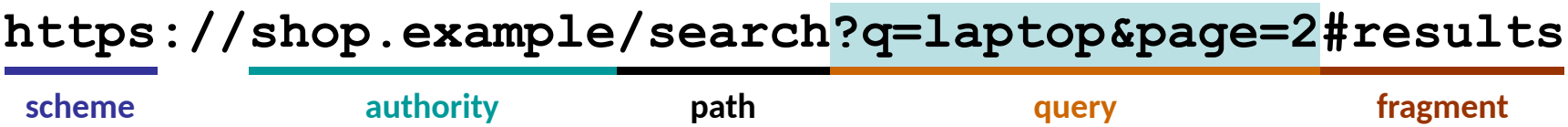
The **method** tells each hop what the client wants to do [RFC 9110 §9.1]

Safe and idempotent methods

Methods	Safe	Idempotent
`GET` `HEAD` `OPTIONS`	yes	yes
`PUT` `DELETE`	no	yes
`POST` `PATCH`	no	no

- **Safe**: the client does not ask for a change. Caches, prefetchers and crawlers can send it [RFC 9110 §9.2.1]
- **Idempotent**: a repeated request has the same intended effect. A client can send it again [RFC 9110 §9.2.2]
- **Neither**: a proxy **must not** send it again automatically [RFC 9110 §9.2.2]
- All safe methods are also idempotent

The parts of a URI



- The **query** and the path together identify the resource [RFC 3986 §3.4]
- The browser does not send the **fragment** to the server [RFC 3986 §3.5]

Two different things, one word

The URI **query component** is part of an identifier. The `QUERY` **method** sends its input as content: the "query content" [RFC 10008 §1.1].

Problems when the query grows

```
the same search, with more filters
GET /search?
q=laptop&brand=lenovo,dell,hp&price_min=40000
&price_max=100000&ram=16,32&sort=price&page=2
&filters=%7B%22gpu%22%3A%5B%22rtx-
4060%22%2C%22rx-7600%22%5D%7D HTTP/1.1
Host: shop.example

HTTP/1.1 414 URI Too Long
```

- **No fixed limit.** HTTP recommends at least 8000 octets [RFC 9110 §4.1]
- **Unknown limits.** Each hop can have a different limit [RFC 10008 §1]
- **414 URI Too Long**: the reply when the URI is too long for the server [RFC 9110 §15.5.15]
- **Encoding cost.** Here, 30 characters of JSON become 54 [RFC 10008 §1]

Where the URI is recorded

Browser history, bookmarks → **CDN / proxy** edge access logs → **WAF** request logs, alerts → **Reverse proxy** access log → **Application** app logs, APM → **SIEM** indexed, kept

access.log (nginx "combined" format)

```
-203.0.113.7 - - [17/Oct/2026:10:42:07 +0530] "GET /users/search?email=alice@example.com&role=admin
 HTTP/1.1" 200 512 "-" "Mozilla/5.0"
+203.0.113.7 - - [17/Oct/2026:10:42:09 +0530] "QUERY /users/search HTTP/1.1" 200 512 "-" "Mozilla/5.0"
```

URI exposure

Access logs, history and bookmarks keep the URI. The `Referer` header sends it on the next same-origin request [RFC 9110 §17.9]

Content exposure

Body logs, WAF and APM tools and each TLS-terminating hop can see the content. Logs keep it less often, *but not never* [RFC 10008 §4]

The gap between GET and POST

	query input in the **URI**	query input in the **content**
Safe and idempotent	**GET** gets a resource; the query is part of the URI	**?** no standard method
Not safe, general processing	`GET /item?do=delete` bad practice: the owner must disallow it [RFC 9110 §9.2.1]	**POST** the resource decides what to do with the content [RFC 9110 §9.3.3]

POST for a query "hinders caching and uses an unsafe method for what would otherwise be a safe request."

[RFC 9110 §17.9]

The gap between GET and POST

	query input in the **URI**	query input in the **content**
Safe and idempotent	**GET** gets a resource; the query is part of the URI	**QUERY** safe and idempotent; the query is in the content [RFC 10008 §2]
Not safe, general processing	`GET /item?do=delete` bad practice: the owner must disallow it [RFC 9110 §9.2.1]	**POST** the resource decides what to do with the content [RFC 9110 §9.3.3]

POST for a query "hinders caching and uses an unsafe method for what would otherwise be a safe request."
[RFC 9110 §17.9]

Not a replacement

QUERY fills the gap. If the query fits in a URI, use GET. If the request changes data, use POST.

2. Objectives

- Explain how HTTP methods tell each hop what a request does
- Show the size and exposure problems of a query in the URI
- Analyse the QUERY method of RFC 10008 with client and server code
- Identify the security effects on browsers, caches, WAFs, gateways and applications
- Review the standards and studies that QUERY depends on
- Recommend when to use GET, QUERY or POST

Authors and Title	Dataset	Purpose of Study	Key Findings
J. Reschke, J. M. Snell, M. Bishop, "The HTTP QUERY Method," RFC 10008, 2026	IETF Standards Track specification	Define a safe, idempotent method that sends the query as request content	Content-Type is mandatory. Responses are cacheable, and the cache key must include the content. New Accept-Query header. Browsers send a CORS preflight
R. Fielding, M. Nottingham, J. Reschke (Eds.), "HTTP Semantics," RFC 9110, 2022	IETF Internet Standard (STD 97)	Define the meaning of HTTP methods, and the safe and idempotent properties	Safe and idempotent describe what the client intends. URIs are not secure. POST for a safe query "uses an unsafe method" (§17.9)
M. Nottingham, "Building Protocols with HTTP," RFC 9205 (BCP 56), 2022	IETF Best Current Practice	Give rules for the design of HTTP-based APIs	Queries in a GET URI can become too long and hard to encode. Queries in POST lose caching and links

Authors and Title	Dataset	Purpose of Study	Key Findings
H. V. Nguyen et al., "Your Cache Has Fallen: Cache-Poisoned Denial-of-Service Attack," ACM CCS, 2019	15 web caching systems (proxy caches and CDNs)	Study attacks that make a cache send an error page to all users	New attack class, CPDoS. 1 proxy cache and 5 CDN services were vulnerable
S. A. Mirheidari et al., "Cached and Confused: Web Cache Deception in the Wild," USENIX Security, 2020	340 high-profile sites (Alexa Top 5K)	Measure web cache deception and its variants	Caches leaked private data and tokens, two years after the attack became public
B. Jabiyev et al., "T-Reqs: HTTP Request Smuggling with Differential Fuzzing," ACM CCS, 2021	10 popular servers, proxies and CDNs, tested in pairs	Find HTTP processing differences that allow request smuggling	Differential fuzzing found new smuggling methods and vulnerable server pairs
J. Chen et al., "We Still Don't Have Secure Cross-Domain Requests: an Empirical Study of CORS," USENIX Security, 2018	Real-world CORS design, implementation and deployment	Measure the security of CORS in practice	CORS adds risky trust relations and is often configured incorrectly

No peer-reviewed study of RFC 10008 itself was found (Oct 2026). These studies examine the mechanisms that QUERY uses.

The same query in three ways (RFC 10008 §1)

GET

```
GET /feed?q=foo&limit=10&sort=-published HTTP/1.1
Host: example.org
```

POST

```
POST /feed HTTP/1.1
Host: example.org
Content-Type: application/x-www-form-urlencoded

q=foo&limit=10&sort=-published
```

QUERY

```
QUERY /feed HTTP/1.1
Host: example.org
Content-Type: application/x-www-form-urlencoded

q=foo&limit=10&sort=-published
```

- **GET** the input is in the URI
- **POST** the input is in the content, but the method does not promise "safe"
- **QUERY** the same content as POST, with a safe and idempotent method

GET, QUERY and POST compared (RFC 10008 Table 1)

	GET	QUERY	POST
Safe	yes	yes	potentially no
Idempotent	yes	yes	potentially no
URI for the query	yes, by definition	optional (Location)	no
URI for the result	optional for all three (Content-Location)		
Cacheable	yes	yes	only for a later GET or HEAD
Request content	no defined meaning	expected	expected

IANA registers QUERY as safe and idempotent [RFC 10008 §5.1]

A QUERY request (RFC 10008 Appendix A.1)

request

```
QUERY /contacts HTTP/1.1 (1)
Host: example.org
Content-Type: application/x-www-form-
urlencoded (2)
Accept: application/json

select=surname,givenname,email&limit=10&match
=%22email=*@example.*%22 (3)
```

1. **QUERY and the target.** The target resource sets the scope of the query [RFC 10008 §2]
2. **Content-Type is mandatory.** If it is missing or wrong, the server must reject the request. It must not guess the type [RFC 10008 §2.1]
3. **The query content.** Here it is form-encoded. The RFC also shows JSON, SQL, JSONPath and XSLT.

A QUERY response (RFC 10008 Appendix A.4)

```
response
HTTP/1.1 200 OK
Content-Type: application/json
Content-Location: /contacts/stored-results/17
  (1)
Location: /contacts/stored-queries/42 (2)
Last-Modified: Sat, 25 Aug 2012 23:34:45 GMT

[ { "surname": "Smith", "givenname": "John",
    "email": "smith@example.org" }, … ]
```

If the query is sensitive, these URIs **should not** contain it [RFC 10008 §4]

1. **Content-Location.** A GET to this URI gets *this result* again [RFC 10008 §2.3]
2. **Location.** A GET to this URI *runs the query* again, with no content [RFC 10008 §2.4]

→ **Redirects.** 301, 302, 307 and 308 keep QUERY. 303 changes it to GET [RFC 10008 §2.5]

✓ **Conditional requests** work as for GET, so 304 is possible [RFC 10008 §2.6]

Send a QUERY request: curl and fetch()

client.sh

```
curl -i -X QUERY localhost:8787/contacts \
  -H 'Content-Type: application/json' \
  --data '{"q":"example.org","limit":10}'
```

output

```
HTTP/1.1 200 OK
Location: /contacts/stored-queries/1
Content-Type: application/json

[{"surname":"Smith","givenname":"John",
 "email":"smith@example.org"}]
```

client.mjs (browser, or Node 24)

```
const url = 'http://localhost:8787/contacts'
const res = await fetch(url, {
  method: 'QUERY', // upper case: not normalised
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ q: 'example', limit: 10 }),
})
console.log(res.status, res.headers.get('Location'))
```

node client.mjs

```
200 /contacts/stored-queries/2
```

Method names are case-sensitive

fetch() changes only DELETE, GET, HEAD, OPTIONS, POST and PUT to upper case [Fetch]. With `method: 'query'` it sends `query`, and the Node 24 server replied **400**.

A server that follows RFC 10008

server.mjs (the handler)

```
14  createServer(async (req, res) => {
15    if (req.method === 'OPTIONS') // discovery, Appendix A.2 and A.3
16      return send(res, 204, null, { Allow, 'Accept-Query': JSON_TYPE })
17    if (req.method === 'GET' && stored.has(req.url)) // GET the Location, §2.4
18      return send(res, 200, search(stored.get(req.url)))
19    if (req.method !== 'QUERY')
20      return send(res, 405, null, { Allow })
21    const type = req.headers['content-type']?.split(';')[0].trim()
22    if (!type) // §2: MUST fail
23      return send(res, 400, { error: 'Content-Type is required' })
24    if (type !== JSON_TYPE) // §2.1 and §3
25      return send(res, 415, null, { 'Accept-Query': JSON_TYPE })
26    let query
27    try { query = JSON.parse(await readBody(req, 16 * 1024)) }
28    catch { return send(res, 400, { error: 'not valid JSON' }) } // no sniffing
29    if (typeof query.q !== 'string') // §2.1: understood, cannot run
30      return send(res, 422, { error: '"q" must be a string' })
31    const location = `/contacts/stored-queries/${stored.size + 1}`
32    stored.set(location, query)
33    send(res, 200, search(query), { Location: location })
34  }).listen(8787, () => console.log('listening on :8787'))
```

- **Lines 15–20**: OPTIONS, a GET to a stored query, and 405 for other methods
- **Lines 22–30**: the checks of [RFC 10008 §2.1] (highlighted)
- **Lines 31–33**: keep the query, and send its URI in Location

Status codes for a QUERY request

Condition	Status	Source
OPTIONS request (discovery)	**204** with Allow, Accept-Query	[RFC 10008 App. A.2]
Content-Type is missing	**400**	[RFC 10008 §2.1]
Media type is not supported	**415** with Accept-Query	[RFC 10008 §2.1]
Content does not agree with the type	**400**; do not guess the type	[RFC 10008 §2.1]
Valid query that the server cannot run	**422**	[RFC 10008 §2.1]
Method not allowed on this resource	**405** with Allow	[RFC 9110 §15.5.6]
Accept value is not supported	**406**	[RFC 10008 §2.1]
Method is not known to the server	**501**	[RFC 9110 §9.1]

The first six rows were tested with curl against `server.mjs`. The last two rows are from the RFCs only.

Safe is not the same as secure

Safe and **idempotent** describe what the client intends. Safety "does not prevent an implementation from including behavior that is potentially harmful" [RFC 9110 §9.2.1].

search-handler.js (excerpt)

```
// excerpt: QUERY content is untrusted input, like any other
async function searchUsers(user, query) {
  if (!user.roles.includes('admin')) // read-only is not "allowed"
    throw new HttpError(403)
  const limit = Math.min(Number(query.limit) || 20, 100) // cost cap
- return db.query(`SELECT email FROM users WHERE email LIKE '%${query.q}%'`)
+ return db.query('SELECT email FROM users WHERE email LIKE $1 LIMIT $2',
+   [`%${query.q}%`, limit], { timeout: 2000 })
}
```

- **Injection**: the query content is untrusted input (SQL, NoSQL, LDAP, search languages)
- **Broken authorization**: read-only does not mean allowed
- **Resource exhaustion**: deep or large queries [Cha et al., 2020]
- **Information disclosure**: results that the caller must not see
- **Automated repetition**: clients repeat safe requests, and so do scrapers

QUERY in a cache: the key must include the content

cache-key.mjs

```
// RFC 10008 §2.7: the key MUST include the content. Run: node cache-key.mjs
import { createHash } from 'node:crypto'

const key = (...parts) =>
  createHash('sha256').update(parts.join('\n')).digest('hex').slice(0, 8)

// allowed: ignore what does not change the meaning (key order)
const canonical = q => JSON.stringify(q, Object.keys(q).sort())
// wrong: keep only the fields this cache knows, so `role` is lost
const lossy = ({ q, limit }) => JSON.stringify({ q, limit })

const admins = { q: 'a', limit: 10, role: 'admin' }
const users = { role: 'user', limit: 10, q: 'a' }

for (const norm of [canonical, lossy]) {
  const k1 = key('QUERY', '/users/search', 'application/json', norm(admins))
  const k2 = key('QUERY', '/users/search', 'application/json', norm(users))
  const verdict = k1 === k2 ? 'SAME KEY' : 'distinct'
  console.log(norm.name.padEnd(10), k1, k2, verdict)
}
```

```
node cache-key.mjs
canonical  40c775a1 e37c99e9 distinct
lossy      85f28a1f 85f28a1f SAME KEY
```

- The key **must** include the content and its metadata [RFC 10008 §2.7]
- Caches **can** normalise the content. A wrong normalisation gives the wrong response [RFC 10008 §4]
- Many caches use only the method and the URI as the key [RFC 9111 §2]
- When the cache and the origin disagree, attackers win [Nguyen 2019] [Mirheidari 2020]

QUERY from a browser: the CORS preflight

```
preflight (browser → server, server → browser)
OPTIONS /contacts HTTP/1.1
Host: api.example
Origin: https://app.example
Access-Control-Request-Method: QUERY
Access-Control-Request-Headers: content-type

HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://app.example
Access-Control-Allow-Methods: QUERY
Access-Control-Allow-Headers: content-type
Access-Control-Expose-Headers: Location, Content-Location
Access-Control-Max-Age: 600
Vary: Origin
```

- A cross-origin QUERY **always** has a preflight. Only GET, HEAD and POST are CORS-safelisted [Fetch] [RFC 10008 §4]
- Same-origin requests, curl and service calls have no preflight
- HTML forms send only GET or POST, and `no-cors` rejects QUERY. Another site cannot send it **without server permission**
- JavaScript can read `Location` only if the server exposes it (line 11)
- A permissive policy with credentials can still leak results [Chen et al., 2018]

Sign a QUERY request (RFC 9421)

```
signed request (Content-Digest of the content shown)
QUERY /contacts HTTP/1.1
Host: api.example
Content-Type: application/json
Content-Digest: sha-256=:GjhZOZY3l3wb+QJHAv9VxkZ0ELmAZyVIh7P1ySNLh/U=:
Signature-Input: sig1=("@method" "@target-uri" "content-type" "content-
digest");created=1792226000;keyid="svc-a"
Signature: sig1=:MEUCIQ…:

{"q":"security","limit":10}
```

Component	Value in a QUERY
`@method`	QUERY
`@target-uri`	https://api.example/contacts
`@query`	only `?` [RFC 9421 §2.2.7]
`content-digest`	the query itself [RFC 9530]

- The signature does **not** cover the content. Sign `content-digest`, and calculate it again [RFC 9421 §7.2.8]
- Some frameworks merge URI and form parameters. An unsigned one can replace a signed one [RFC 9421 §7.5.8]
- A signature gives integrity and origin. It does not give authorization or confidentiality

Deploy QUERY: all hops must support it

Hop	Must do for QUERY	If it does not
Browser, client	preflight; upper-case `QUERY`; exposed headers	request blocked, or sent as `query`
CDN, cache	key on method, URI, content and type; correct normalisation	wrong or poisoned responses
WAF	allow it; examine its body as a POST body; size limits	a change of method avoids the rules [OWASP WSTG-CONF-06]
Load balancer, proxy	forward it; frame the body the same way as the next hop	405 or 501, or request smuggling [Jabiyev 2021]
Framework, app	route it; strict Content-Type; authorization, validation, cost limits	415 / 422 gaps, injection, DoS
Database	parameterised queries, timeouts, result limits	injection, exhaustion
Logs, SIEM, docs	parse the method; no raw sensitive bodies; OpenAPI 3.2 has `query` [OAS 3.2]	gaps in monitoring

Conclusion

GET

Get a resource by its URI

- the query fits in a URI
- links and caches are important

QUERY

Safe query with content

- the operation *is* a query
- the input is large or structured
- all hops support it

POST

General processing

- the operation is not safe
- it changes data or submits a form

- **QUERY does not replace GET or POST.** It fills one gap: safe, idempotent queries with request content.
- Its security depends on how clients, caches, WAFs, gateways, applications and logs **implement** it. The method alone gives no security.

References

[1] J. Reschke, J. M. Snell, and M. Bishop, “The HTTP QUERY Method,” RFC 10008, IETF, Jun. 2026, doi: 10.17487/RFC10008.

[2] R. Fielding, M. Nottingham, and J. Reschke, Eds., “HTTP Semantics,” STD 97, RFC 9110, Jun. 2022.

[3] R. Fielding, M. Nottingham, and J. Reschke, Eds., “HTTP Caching,” STD 98, RFC 9111, Jun. 2022.

[4] R. Fielding, M. Nottingham, and J. Reschke, Eds., “HTTP/1.1,” STD 99, RFC 9112, Jun. 2022.

[5] T. Berners-Lee, R. Fielding, and L. Masinter, “Uniform Resource Identifier (URI): Generic Syntax,” STD 66, RFC 3986, Jan. 2005.

[6] M. Nottingham, “Building Protocols with HTTP,” BCP 56, RFC 9205, Jun. 2022.

[7] A. Backman, J. Richer, and M. Sporny, Eds., “HTTP Message Signatures,” RFC 9421, Feb. 2024.

[8] R. Polli and L. Pardue, “Digest Fields,” RFC 9530, Feb. 2024.

[9] WHATWG, “Fetch Standard” and “HTML Standard,” Living Standards.

[10] W3C, “Referrer Policy.”

[11] IANA, “Hypertext Transfer Protocol (HTTP) Method Registry.”

[12] OpenAPI Initiative, “OpenAPI Specification v3.2.0,” Sep. 2025.

[13] OWASP, “WSTG-CONF-06: Test HTTP Methods,” Web Security Testing Guide.

[14] MDN Web Docs, “Cross-Origin Resource Sharing (CORS).”

References

[15] O. Starov, P. Gill, and N. Nikiforakis, “Are You Sure You Want to Contact Us? Quantifying the Leakage of PII via Website Contact Forms,” Proc. Privacy Enhancing Technologies, vol. 2016, no. 1, pp. 20–33, 2016, doi: 10.1515/popets-2015-0028.

[16] H. V. Nguyen, L. Lo Iacono, and H. Federrath, “Your Cache Has Fallen: Cache-Poisoned Denial-of-Service Attack,” in Proc. ACM CCS, 2019, pp. 1915–1936, doi: 10.1145/3319535.3354215.

[17] S. A. Mirheidari, S. Arshad, K. Onarlioglu, B. Crispo, E. Kirda, and W. Robertson, “Cached and Confused: Web Cache Deception in the Wild,” in Proc. 29th USENIX Security Symposium, 2020.

[18] B. Jabiyev, S. Sprecher, K. Onarlioglu, and E. Kirda, “T-Reqs: HTTP Request Smuggling with Differential Fuzzing,” in Proc. ACM CCS, 2021, pp. 1805–1820, doi: 10.1145/3460120.3485384.

[19] J. Chen, J. Jiang, H. Duan, T. Wan, S. Chen, V. Paxson, and M. Yang, “We Still Don’t Have Secure Cross-Domain Requests: an Empirical Study of CORS,” in Proc. 27th USENIX Security Symposium, 2018.

[20] A. Cha, E. Wittern, G. Baudart, J. C. Davis, L. Mandel, and J. A. Laredo, “A Principled Approach to GraphQL Query Cost Analysis,” in Proc. ESEC/FSE, 2020, pp. 257–268, doi: 10.1145/3368089.3409670.

[21] O. Hartig and J. Pérez, “Semantics and Complexity of GraphQL,” in Proc. The Web Conference (WWW), 2018, pp. 1155–1164, doi: 10.1145/3178876.3186014.

[22] J. Chen, J. Jiang, H. Duan, N. Weaver, T. Wan, and V. Paxson, “Host of Troubles: Multiple Host Ambiguities in HTTP Implementations,” in Proc. ACM CCS, 2016, pp. 1516–1527, doi: 10.1145/2976749.2978394.

Queries

&

Suggestions

THANK YOU...